

PATRONES DE CREACIÓN

Singleton y Factory Method

Objetivo

Entender qué problema resuelve cada patrón, por qué se utiliza y cómo implementarlo en Java antes de resolver ejercicios.

0. Antes de empezar: la idea general

Los patrones de creación se ocupan de cómo se crean los objetos de un sistema. La idea es evitar que la creación quede repartida y mezclada con el resto de la lógica del programa.

Qué debes aprender de estos dos patrones

No se trata de memorizar código. Debes saber reconocer el problema: Singleton controla que exista una única instancia; Factory Method separa el uso de un objeto de la decisión sobre qué clase concreta hay que crear.

Dos preguntas para orientarte

Patrón	Pregunta que resuelve
Singleton	¿Cómo consigo que exista un único objeto de una clase y que todos utilicen ese mismo objeto?
Factory Method	¿Cómo evito que el código general tenga que decidir qué clase concreta crear?

Chuleta rápida

Singleton = controla CUÁNTOS objetos se crean.

Factory Method = controla QUIÉN decide QUÉ objeto concreto se crea.

1. SINGLETON - una única instancia compartida

1.1. ¿Qué problema resuelve?

Imagina una aplicación con varias pantallas. Todas deben utilizar la misma configuración: tema claro/oscuro, idioma, tamaño de letra, etc. No tendría sentido que cada pantalla tuviera una configuración independiente si queremos un comportamiento global.

Definición

Singleton significa: “de esta clase quiero que exista UN SOLO objeto en todo el programa y quiero poder acceder siempre a ese mismo objeto”.

1.2. ¿Cómo se consigue?

La implementación básica se puede recordar en tres pasos:

1. Impedimos que desde fuera se puedan crear objetos libremente: el constructor es private.
2. La propia clase guarda una referencia a su única instancia en una variable static.

3. La clase ofrece un método static que devuelve esa instancia y la crea solo la primera vez.

```
class MiSingleton {  
    private static MiSingleton instancia;  
  
    private MiSingleton() {  
    }  
  
    public static MiSingleton getInstancia() {  
        if (instancia == null) {  
            instancia = new MiSingleton();  
        }  
        return instancia;  
    }  
}
```

¿Por qué el constructor es private?

Porque así otras clases no pueden hacer `new MiSingleton()`. La clase se reserva para sí misma el control sobre la creación del objeto.

¿Por qué la instancia es static?

Porque pertenece a la clase, no a un objeto concreto. Podemos acceder a ella sin haber creado previamente un objeto.

¿Qué hace `getInstancia()`?

Comprueba si el objeto ya existe. Si no existe, lo crea. Si ya existe, devuelve el mismo. Por eso todas las partes del programa terminan utilizando una única instancia.

1.3. Ejemplo resuelto: Gestor de tema

Queremos que todas las pantallas de una aplicación utilicen el mismo tema visual.

```
class GestorTema {

    private static GestorTema instancia;
    private String tema = "CLARO";

    private GestorTema() {
    }

    public static GestorTema getInstancia() {
        if (instancia == null) {
            instancia = new GestorTema();
        }
        return instancia;
    }

    public void setTema(String tema) {
        this.tema = tema;
    }

    public String getTema() {
        return tema;
    }
}

class PantallaInicio {
    public void mostrar() {
        GestorTema gestor = GestorTema.getInstancia();
        System.out.println("Pantalla Inicio. Tema: " + gestor.getTema());
    }
}

class PantallaPerfil {
    public void mostrar() {
        GestorTema gestor = GestorTema.getInstancia();
        System.out.println("Pantalla Perfil. Tema: " + gestor.getTema());
    }
}

public class Main {
    public static void main(String[] args) {
        GestorTema gestor = GestorTema.getInstancia();
        gestor.setTema("OSCURO");

        new PantallaInicio().mostrar();
        new PantallaPerfil().mostrar();
    }
}
```

¿Qué demuestra el ejemplo?

Las dos pantallas muestran OSCURO porque ambas están accediendo al mismo GestorTema. No existe un GestorTema distinto por pantalla.

1.4. Ejercicio para entregar - Singleton

EJERCICIO 1: Configuración de una interfaz

Crea un gestor de configuración compartido por diferentes pantallas de una aplicación. Debe existir una única ConfiguracionInterfaz en todo el programa.

Requisitos:

- Crear la clase ConfiguracionInterfaz siguiendo el patrón Singleton.
- Guardar dos datos: idioma (String) y tamañoFuente (int).
- Valores iniciales sugeridos: idioma = "ES" y tamañoFuente = 16.
- Crear las clases PantallaInicio y PantallaAjustes, cada una con un método mostrar().
- Cada pantalla debe obtener ConfiguracionInterfaz mediante getInstance() y mostrar la configuración actual.
- En Main, cambiar la configuración a idioma = "EN" y tamañoFuente = 20 antes de mostrar las dos pantallas.

No es necesario utilizar una biblioteca gráfica. La interfaz se representa con mensajes por consola.

Pregunta que debes poder responder

¿Por qué PantallaInicio y PantallaAjustes ven exactamente la misma configuración aunque cada una llame a getInstance() por separado?

2. FACTORY METHOD - delegar la decisión de qué objeto crear

2.1. El problema antes del patrón

Imagina ahora una aplicación que puede ejecutarse en móvil y escritorio. En ambos entornos necesitamos botones, pero el botón concreto puede ser diferente.

```
if (tipo.equals("movil")) {  
    boton = new BotonMovil();  
} else {  
    boton = new BotonEscritorio();  
}
```

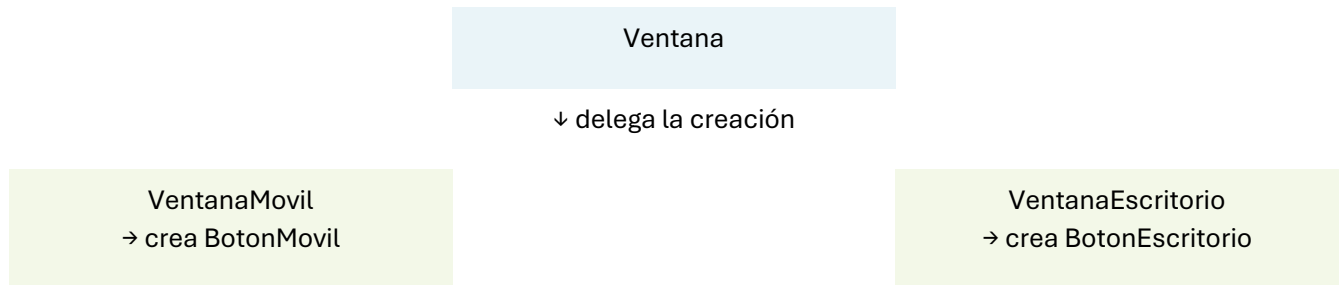
Este código funciona. El problema aparece cuando el programa crece y muchas partes empiezan a decidir qué clase concreta deben instanciar. Entonces el código general queda demasiado acoplado a BotonMovil, BotonEscritorio, BotonWeb, etc.

Definición

Factory Method permite que una clase UTILICE objetos sin tener que decidir exactamente qué clase concreta debe CREAR. Esa decisión se deja a sus subclases.

2.2. ¿Qué significa “delegar en las subclases la decisión de qué objeto crear”?

Delegar significa: “yo necesito el objeto, pero no voy a decidir aquí qué clase concreta se crea; otra clase se encargará de tomar esa decisión”.



La clase padre Ventana dice: “necesito un botón, pero no sé ni quiero saber si será móvil o de escritorio”.

Las subclases responden: VentanaMovil crea BotonMovil; VentanaEscritorio crea BotonEscritorio.

¿Por qué hacerlo así?

Porque queremos separar el código que USA un botón del código que DECIDE qué botón concreto crear. La lógica general queda más limpia y depende menos de clases concretas.

2.3. ¿Qué significa “compartir una interfaz común”?

Creamos una interfaz que define qué operaciones deben ofrecer todos los productos que queremos tratar de la misma manera:

```
interface Boton {
    void dibujar();
}
```

Después, diferentes clases concretas implementan ese mismo contrato:

```
class BotonMovil implements Boton {
    public void dibujar() {
        System.out.println("Dibujando botón móvil");
    }
}

class BotonEscritorio implements Boton {
    public void dibujar() {
        System.out.println("Dibujando botón de escritorio");
    }
}
```

Interfaz común = contrato común

BotonMovil y BotonEscritorio son clases diferentes, pero ambas prometen que tienen dibujar(). Por eso el resto del programa puede trabajar con una variable de tipo Boton sin necesitar saber cuál de las dos clases concretas tiene delante.

Esto permite escribir:

```
Boton boton = crearBoton();
boton.dibujar();
```

La clase que utiliza el botón solo necesita saber que recibe un Boton. No necesita preguntar si es BotonMovil o BotonEscritorio.

2.4. ¿Por qué las subclases redefinen el método de creación?

La clase padre declara el método que todos los creadores deberán tener:

```
public abstract Boton crearBoton();
```

Pero no lo implementa porque precisamente quiere dejar abierta la decisión sobre qué producto concreto se crea.

Cada subclase redefine ese mismo método y toma una decisión diferente:

```
class VentanaMovil extends Ventana {
    @Override
    public Boton crearBoton() {
        return new BotonMovil();
    }
}

class VentanaEscritorio extends Ventana {
    @Override
    public Boton crearBoton() {
        return new BotonEscritorio();
    }
}
```

La clave

El método se llama igual y devuelve el mismo tipo general (Boton), pero cada subclase decide qué clase concreta instanciar. Eso es “redefinir el método de creación”.

2.5. ¿Qué beneficios obtenemos?

El código general conoce menos clases concretas.

Ventana trabaja con Boton, no necesita conocer todos los tipos de botón.

Cada clase tiene una responsabilidad clara.

Ventana utiliza botones; las subclases creadoras deciden cuál corresponde.

Añadir nuevos tipos resulta más sencillo.

Podemos añadir BotonWeb + VentanaWeb sin modificar la lógica general de Ventana.

Se reduce el acoplamiento.

La lógica principal depende de una abstracción común y no de cada implementación concreta.

Frase que debes recordar

Factory Method no consiste simplemente en crear objetos. Consiste en decidir **DÓNDE** colocamos la decisión de qué objeto concreto se crea.

2.6. Las cuatro piezas del patrón

Pieza	En nuestro ejemplo	Qué hace
Producto	Boton (interfaz)	Define el contrato común.
Productos concretos	BotonMovil / BotonEscritorio	Implementan el contrato de maneras distintas.
Creador	Ventana	Declara crearBoton() y utiliza el producto sin conocer su clase concreta.
Creadores concretos	VentanaMovil / VentanaEscritorio	Redefinen crearBoton() y deciden qué producto concreto instanciar.

2.7. Ejemplo resuelto: botones de una interfaz

```
interface Boton {
    void dibujar();
}

class BotonMovil implements Boton {
    @Override
    public void dibujar() {
        System.out.println("Dibujando botón para MÓVIL");
    }
}

class BotonEscritorio implements Boton {
    @Override
    public void dibujar() {
        System.out.println("Dibujando botón para ESCRITORIO");
    }
}

abstract class Ventana {

    // Factory Method
    public abstract Boton crearBoton();

    public void mostrar() {
        Boton boton = crearBoton();
        boton.dibujar();
    }
}
```

```

class VentanaMovil extends Ventana {
    @Override
    public Boton crearBoton() {
        return new BotonMovil();
    }
}

class VentanaEscritorio extends Ventana {
    @Override
    public Boton crearBoton() {
        return new BotonEscritorio();
    }
}

public class Main {
    public static void main(String[] args) {
        Ventana movil = new VentanaMovil();
        movil.mostrar();

        Ventana escritorio = new VentanaEscritorio();
        escritorio.mostrar();
    }
}

```

Qué debes observar

Ventana.mostrar() utiliza un Boton, pero nunca hace new BotonMovil() ni new BotonEscritorio(). La decisión concreta está en las subclases creadoras.

2.8. Ejercicio para entregar - Factory Method

EJERCICIO 2: Campos de texto para móvil y escritorio

Una aplicación debe mostrar campos de texto adaptados a dos entornos. Debes utilizar Factory Method para que el código general no decida directamente qué clase concreta de CampoTexto crear.

Requisitos:

- Crear la interfaz CampoTexto con el método void dibujar().
- Crear CampoTextoMovil y CampoTextoEscritorio, ambas implementando CampoTexto.
- CampoTextoMovil debe mostrar: “Dibujando campo de texto para MÓVIL”.
- CampoTextoEscritorio debe mostrar: “Dibujando campo de texto para ESCRITORIO”.
- Crear una clase abstracta Formulario que declare public abstract CampoTexto crearCampoTexto().
- Formulario debe tener un método mostrar() que llame a crearCampoTexto() y después a dibujar().
- Crear FormularioMovil y FormularioEscritorio redefiniendo crearCampoTexto().
- En Main, crear un FormularioMovil y un FormularioEscritorio y llamar a mostrar().

Restricción importante

En Main NO se permite hacer new CampoTextoMovil() ni new CampoTextoEscritorio(). Esos objetos deben ser creados a través del Factory Method.

Preguntas que debes poder responder

- 1) ¿Qué papel desempeña la interfaz CampoTexto?
- 2) ¿Qué significa aquí “delegar” la creación?
- 3) ¿Por qué FormularioMovil y FormularioEscritorio redefinen crearCampoTexto()?
- 4) Si mañana añadimos una versión Web, ¿qué clases nuevas crearías y qué código existente no tendrías que modificar?

3. Resumen final y checklist**3.1. Singleton vs Factory Method**

Aspecto	Singleton	Factory Method
Pregunta principal	¿Cuántas instancias permito?	¿Quién decide qué clase concreta se crea?
Idea clave	Una sola instancia compartida.	La creación concreta se delega en subclases.
Elemento importante	Constructor private + instancia static + getInstance().	Producto común + creador + creadores concretos.
Uso en nuestros ejercicios	Configuración compartida por pantallas.	Componentes de interfaz distintos según el entorno.
Error conceptual típico	Crear objetos con new desde fuera.	Pensar que Factory Method es solo “una clase fábrica con if”.

3.2. Checklist antes de entregar

- Puedo explicar con mis palabras qué problema resuelve cada patrón.
- En Singleton, el constructor es private y no creo instancias con new desde otras clases.
- En Singleton, todas las pantallas acceden al mismo objeto mediante getInstance().
- En Factory Method, los productos concretos comparten una interfaz o tipo común.
- La clase creadora utiliza el tipo común y no necesita conocer la clase concreta.
- Cada creador concreto redefine el método de fabricación y decide qué producto crea.
- Puedo explicar qué beneficio aporta separar “usar” de “crear”.
- Mi Main prueba el comportamiento, pero no contiene la lógica de creación que el patrón pretende encapsular.

Objetivo de aprendizaje

Al terminar los ejercicios no basta con que el programa funcione. Debes ser capaz de señalar en tu código dónde está el patrón y explicar por qué esa estructura resuelve el problema planteado.